

Machine Learning for Nuclear Physics and Many-body Physics

Morten Hjorth-Jensen¹

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway¹

Center for Computational Sciences, University of Tsukuba,
February 20, 2026

What is this talk about?

The main emphasis is to give you a short and pedestrian introduction to the whys and hows we can use (with several examples) machine learning methods to solve quantum mechanical many-body problems. And why this could (or should) be of interest.

Recent preprint (PPNN submitted)

Lovato, Carleo, Fore, MHJ, Jane Kim, Rios, Rocco,
Neural-network quantum states for the nuclear many-body problem, see <https://arxiv.org/abs/2602.13826>

Thanks to many

Jane Kim (ANL), Patrick Cook (MSU), Danny Jammooa (MSU/Riken), Dean Lee (MSU), Even Nordhagen (UiO), Aleksander Sekkelsten (UiO), Bryce Fore (ANL), Alessandro Lovato (ANL), Stefano Gandolfi (LANL), Francesco Pederiva (UniTN), Arnau Rios (Barcelona), and Giuseppe Carleo (EPFL). Excuses to those I have omitted/forgotten.

AI/ML and some statements you may have heard (and what do they mean?)

1. Fei-Fei Li on ImageNet: **map out the entire world of objects** ([The data that transformed AI research](#))
2. Russell and Norvig in their popular textbook: **relevant to any intellectual task; it is truly a universal field** ([Artificial Intelligence, A modern approach](#))
3. Woody Bledsoe puts it more bluntly: **in the long run, AI is the only science** (quoted in Pamilla McCorduck, [Machines who think](#))

If you wish to have a critical read on AI/ML from a societal point of view, see [Kate Crawford's recent text Atlas of AI](#).

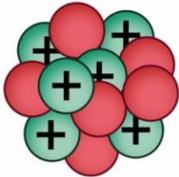
Here: with AI/ML we intend a collection of machine learning methods with an emphasis on statistical learning and data analysis

Quantum mechanical many-body problems (nuclear example here)

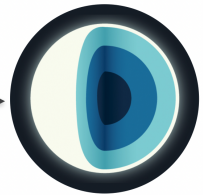
Many-body Schrödinger equation

$$\left(-\sum_i \frac{\nabla_i^2}{2m_N} + V \right) |\Psi_0\rangle = E_0 |\Psi_0\rangle$$

Nuclei



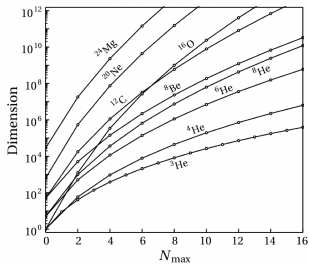
Neutron stars



Curse of dimensionality

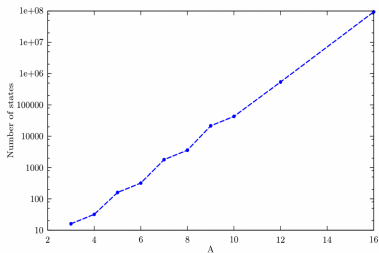
Configuration-interaction

$$\binom{N}{A} = \frac{N!}{(N-A)!A!}$$



Green's function Monte Carlo

$$\lim_{\tau \rightarrow \infty} e^{-(H-E_0)\tau} |\Psi_T\rangle = c_0 |\Psi_0\rangle$$



Neural network quantum states

Neural networks compactly represent complex high-dimensional functions

Most quantum states of interest have distinctive features and intrinsic structures



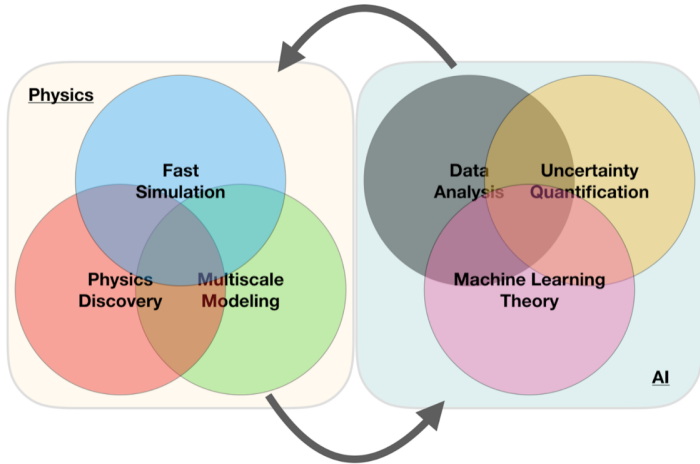
Hilbert Space

Physical States

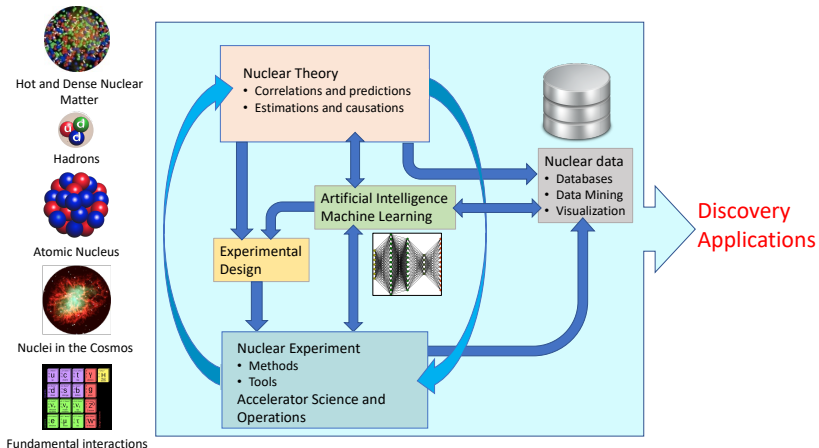
Mean-field

Credit: Giuseppe Carleo

Machine learning. A simple perspective on the interface between ML and Physics



ML in Nuclear Physics (or any field, almost)



Types of machine learning

The approaches to machine learning are many, but are often split into two main categories. In *supervised learning* we know the answer to a problem, and let the computer deduce the logic behind it. On the other hand, *unsupervised learning* is a method for finding patterns and relationship in data sets without any prior knowledge of the system.

An important third category is *reinforcement learning*. This is a paradigm of learning inspired by behavioural psychology, where learning is achieved by trial-and-error, solely from rewards and punishment.

Main categories

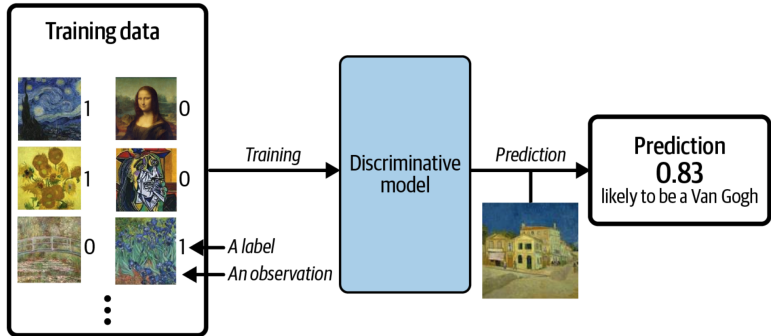
Another way to categorize machine learning tasks is to consider the desired output of a system. Some of the most common tasks are:

- ▶ **Classification:** Outputs are divided into two or more classes. The goal is to produce a model that assigns inputs into one of these classes. An example is to identify digits based on pictures of hand-written ones. Classification is typically supervised learning.
- ▶ **Regression:** Finding a functional relationship between an input data set and a reference data set. The goal is to construct a function that maps input data to continuous output values.
- ▶ **Clustering:** Data are divided into groups with certain common traits, without knowing the different groups beforehand. It is thus a form of unsupervised learning.

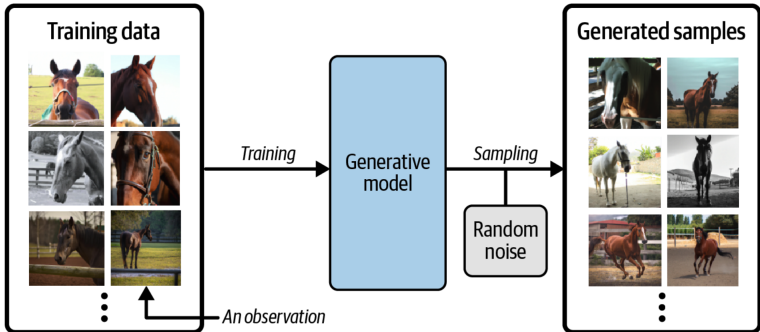
The plethora of machine learning algorithms/methods

1. Deep learning: Neural Networks (NN), Convolutional NN, Recurrent NN, Boltzmann machines, autoencoders and variational autoencoders and generative adversarial networks, stable diffusion and many more generative models
2. Bayesian statistics and Bayesian Machine Learning, Bayesian experimental design, Bayesian Regression models, Bayesian neural networks, Gaussian processes and much more
3. Dimensionality reduction (Principal component analysis), Clustering Methods and more
4. Ensemble Methods, Random forests, bagging and voting methods, gradient boosting approaches
5. Linear and logistic regression, Kernel methods, support vector machines and more
6. Reinforcement Learning; Transfer Learning and more

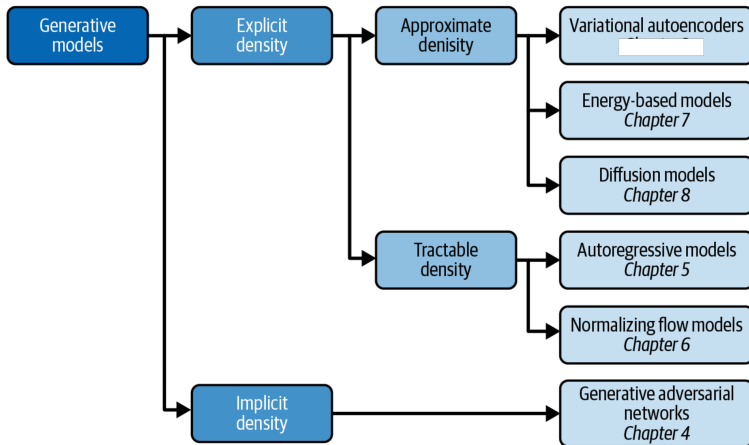
Example of discriminative modeling, taken from Generative Deep Learning by David Foster



Example of generative modeling, taken from Generative Deep Learning by David Foster



Taxonomy of generative deep learning, taken from Generative Deep Learning by David Foster



Good books with hands-on material and codes

- ▶ Sebastian Rashcka et al, Machine learning with Sickit-Learn and PyTorch
- ▶ David Foster, Generative Deep Learning with TensorFlow
- ▶ Babcock and Gavras, Generative AI with Python and TensorFlow 2

All three books have GitHub sites from where one can download all codes. A good and more general text (2016) is Goodfellow, Bengio and Courville, [Deep Learning](#)

More references

Reading on diffusion models

1. A central paper is the one by Sohl-Dickstein et al, Deep Unsupervised Learning using Nonequilibrium Thermodynamics, <https://arxiv.org/abs/1503.03585>
2. See also Diederik P. Kingma, Tim Salimans, Ben Poole, Jonathan Ho, Variational Diffusion Models, <https://arxiv.org/abs/2107.00630>

and VAEs

1. An Introduction to Variational Autoencoders, by Kingma and Welling, see <https://arxiv.org/abs/1906.02691>

And two Nobel prizes in 2024 (Chemistry and Physics!

What are the basic Machine Learning ingredients?

Almost every problem in ML and data science starts with the same ingredients:

- ▶ The dataset $\mathbf{x}$ (could be some observable quantity of the system we are studying)
- ▶ A model which is a function of a set of parameters α that relates to the dataset, say a likelihood function $p(\mathbf{x}|\alpha)$ or just a simple model $f(\alpha)$
- ▶ A so-called **loss/cost/risk** function $\mathcal{C}(\mathbf{x}, f(\alpha))$ which allows us to decide how well our model represents the dataset.

We seek to minimize the function $\mathcal{C}(\mathbf{x}, f(\alpha))$ by finding the parameter values which minimize $\mathcal{C}$. This leads to various minimization algorithms. It may surprise many, but at the heart of all machine learning algorithms there is an optimization problem.

Low-level machine learning, the family of ordinary least squares methods

Our data which we want to apply a machine learning method on, consist of a set of inputs $\mathbf{x}^T = [x_0, x_1, x_2, \dots, x_{n-1}]$ and the outputs we want to model $\mathbf{y}^T = [y_0, y_1, y_2, \dots, y_{n-1}]$. We assume that the output data can be represented (for a regression case) by a continuous function f through

$$\mathbf{y} = f(\mathbf{x}) + \epsilon.$$

Setting up the equations

In linear regression we approximate the unknown function with another continuous function $\tilde{\mathbf{y}}(\mathbf{x})$ which depends linearly on some unknown parameters $\boldsymbol{\theta}^T = [\theta_0, \theta_1, \theta_2, \dots, \theta_{p-1}]$.

The input data can be organized in terms of a so-called design matrix with an approximating function $\tilde{\mathbf{y}}$

$$\tilde{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta},$$

The objective/cost/loss function

The simplest approach is the mean squared error

$$C(\Theta) = \frac{1}{n} \sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2 = \frac{1}{n} \left\{ (\mathbf{y} - \tilde{\mathbf{y}})^T (\mathbf{y} - \tilde{\mathbf{y}}) \right\},$$

or using the matrix $\mathbf{X}$ and in a more compact matrix-vector notation as

$$C(\Theta) = \frac{1}{n} \left\{ (\mathbf{y} - \mathbf{X}\theta)^T (\mathbf{y} - \mathbf{X}\theta) \right\}.$$

This function represents one of many possible ways to define the so-called cost function.

Training solution

Optimizing with respect to the unknown parameters θ_j we get

$$\mathbf{X}^T \mathbf{y} = \mathbf{X}^T \mathbf{X} \boldsymbol{\theta},$$

and if the matrix $\mathbf{X}^T \mathbf{X}$ is invertible we have the optimal values

$$\hat{\boldsymbol{\theta}} = \left(\mathbf{X}^T \mathbf{X} \right)^{-1} \mathbf{X}^T \mathbf{y}.$$

We say we 'learn' the unknown parameters $\boldsymbol{\theta}$ from the last equation.

Selected references

- ▶ Mehta et al. and Physics Reports (2019).
- ▶ Machine Learning and the Physical Sciences by Carleo et al
- ▶ Artificial Intelligence and Machine Learning in Nuclear Physics, Amber Boehnlein et al., Reviews Modern of Physics 94, 031003 (2022)
- ▶ Dilute neutron star matter from neural-network quantum states by Fore et al, Physical Review Research 5, 033062 (2023)
- ▶ Neural-network quantum states for ultra-cold Fermi gases, Jane Kim et al, Commun Phys 7, 148 (2024), see <https://doi.org/10.48550/arXiv.2305.08831>

Scientific Machine Learning

An important and emerging field is what has been dubbed as scientific ML, see the article by Deiana et al "Applications and Techniques for Fast Machine Learning in Science, Big Data **5**, 787421 (2022):<https://doi.org/10.3389/fdata.2022.787421>"

The authors discuss applications and techniques for fast machine learning (ML) in science – the concept of integrating power ML methods into the real-time experimental data processing loop to accelerate scientific discovery. The report covers three main areas

1. applications for fast ML across a number of scientific domains;
2. techniques for training and implementing performant and resource-efficient ML algorithms;
3. and computing architectures, platforms, and technologies for deploying these algorithms.

And more

- ▶ An important application of AI/ML methods is to improve the estimation of bias or uncertainty due to the introduction of or lack of physical constraints in various theoretical models.
- ▶ In theory, we expect to use AI/ML algorithms and methods to improve our knowledge about correlations of physical model parameters in data for quantum many-body systems. Deep learning methods show great promise in circumventing the exploding dimensionalities encountered in quantum mechanical many-body studies.
- ▶ Merging a frequentist approach (the standard path in ML theory) with a Bayesian approach, has the potential to infer better probability distributions and error estimates.
- ▶ Machine Learning and Quantum Computing is a very interesting avenue

Many-body physics, Quantum Monte Carlo and deep learning

Given a hamiltonian H and a trial wave function Ψ_T , the variational principle states that the expectation value of $\langle H \rangle$, defined through

$$\langle E \rangle = \frac{\int d\mathbf{R} \Psi_T^*(\mathbf{R}) H(\mathbf{R}) \Psi_T(\mathbf{R})}{\int d\mathbf{R} \Psi_T^*(\mathbf{R}) \Psi_T(\mathbf{R})},$$

is an upper bound to the ground state energy E_0 of the hamiltonian H , that is

$$E_0 \leq \langle E \rangle.$$

In general, the integrals involved in the calculation of various expectation values are multi-dimensional ones. Traditional integration methods such as the Gauss-Legendre will not be adequate for say the computation of the energy of a many-body system. **Basic philosophy: Let a neural network find the optimal wave function**

Quantum Monte Carlo Motivation

Basic steps

Choose a trial wave function $\psi_T(\mathbf{R})$.

$$P(\mathbf{R}, \alpha) = \frac{|\psi_T(\mathbf{R}, \alpha)|^2}{\int |\psi_T(\mathbf{R}, \alpha)|^2 d\mathbf{R}}.$$

This is our model, or likelihood/probability distribution function (PDF). It depends on some variational parameters α . The approximation to the expectation value of the Hamiltonian is now

$$\langle E[\alpha] \rangle = \frac{\int d\mathbf{R} \psi_T^*(\mathbf{R}, \alpha) H(\mathbf{R}) \psi_T(\mathbf{R}, \alpha)}{\int d\mathbf{R} \psi_T^*(\mathbf{R}, \alpha) \psi_T(\mathbf{R}, \alpha)}.$$

Quantum Monte Carlo Motivation

Define a new quantity

$$E_L(\mathbf{R}, \alpha) = \frac{1}{\psi_T(\mathbf{R}, \alpha)} H \psi_T(\mathbf{R}, \alpha),$$

called the local energy, which, together with our trial PDF yields

$$\langle E[\alpha] \rangle = \int P(\mathbf{R}) E_L(\mathbf{R}, \alpha) d\mathbf{R} \approx \frac{1}{N} \sum_{i=1}^N E_L(\mathbf{R}_i, \alpha)$$

with N being the number of Monte Carlo samples.

Energy derivatives

The local energy as function of the variational parameters defines now our **objective/cost** function.

To find the derivatives of the local energy expectation value as function of the variational parameters, we can use the chain rule and the hermiticity of the Hamiltonian.

Let us define (with the notation $\langle E[\alpha] \rangle = \langle E_L \rangle$)

$$\bar{E}_{\alpha_i} = \frac{d\langle E_L \rangle}{d\alpha_i},$$

as the derivative of the energy with respect to the variational parameter α_i . We define also the derivative of the trial function (skipping the subindex T) as

$$\bar{\Psi}_i = \frac{d\Psi}{d\alpha_i}.$$

Derivatives of the local energy

The elements of the gradient of the local energy are

$$\bar{E}_i = 2 \left(\left\langle \frac{\bar{\Psi}_i}{\Psi} E_L \right\rangle - \left\langle \frac{\bar{\Psi}_i}{\Psi} \right\rangle \langle E_L \rangle \right).$$

From a computational point of view it means that you need to compute the expectation values of

$$\left\langle \frac{\bar{\Psi}_i}{\Psi} E_L \right\rangle,$$

and

$$\left\langle \frac{\bar{\Psi}_i}{\Psi} \right\rangle \langle E_L \rangle$$

These integrals are evaluated using MC integration (with all its possible error sources). Use methods like stochastic gradient or other minimization methods to find the optimal parameters.

Why Feed Forward Neural Networks (FFNN)?

According to the *Universal approximation theorem*, a feed-forward neural network with just a single hidden layer containing a finite number of neurons can approximate a continuous multidimensional function to arbitrary accuracy.

Universal approximation theorem

The universal approximation theorem plays a central role in deep learning. Cybenko (1989) showed the following:

Let σ be any continuous sigmoidal function such that

$$\sigma(z) = \begin{cases} 1 & z \rightarrow \infty \\ 0 & z \rightarrow -\infty \end{cases}$$

Given a continuous and deterministic function $F(\mathbf{x})$ on the unit cube in d -dimensions $F \in [0, 1]^d$, $\mathbf{x} \in [0, 1]^d$ and a parameter $\epsilon > 0$, there is a one-layer (hidden) neural network $f(\mathbf{x}; \Theta)$ with $\Theta = (\mathbf{W}, \mathbf{b})$ and $\mathbf{W} \in \mathbb{R}^{m \times n}$ and $\mathbf{b} \in \mathbb{R}^n$, for which

$$|F(\mathbf{x}) - f(\mathbf{x}; \Theta)| < \epsilon \quad \forall \mathbf{x} \in [0, 1]^d.$$

The approximation theorem in words

Any continuous function $y = F(\mathbf{x})$ supported on the unit cube in d -dimensions can be approximated by a one-layer sigmoidal network to arbitrary accuracy.

[Hornik \(1991\)](#) extended the theorem by letting any non-constant, bounded activation function to be included using that the expectation value

$$\mathbb{E}[|F(\mathbf{x})|^2] = \int_{\mathbf{x} \in D} |F(\mathbf{x})|^2 p(\mathbf{x}) d\mathbf{x} < \infty.$$

Then we have

$$\mathbb{E}[|F(\mathbf{x}) - f(\mathbf{x}; \Theta)|^2] = \int_{\mathbf{x} \in D} |F(\mathbf{x}) - f(\mathbf{x}; \Theta)|^2 p(\mathbf{x}) d\mathbf{x} < \epsilon.$$

More on the general approximation theorem

None of the proofs give any insight into the relation between the number of hidden layers and nodes and the approximation error ϵ , nor the magnitudes of $\mathbf{W}$ and $\mathbf{b}$.

Neural networks (NNs) have what we may call a kind of universality no matter what function we want to compute.

It does not mean that an NN can be used to exactly compute any function. Rather, we get an approximation that is as good as we want.

Class of functions we can approximate

The class of functions that can be approximated are the continuous ones. If the function $F(\mathbf{x})$ is discontinuous, it won't in general be possible to approximate it. However, an NN may still give an approximation even if we fail in some points.

Monte Carlo methods and Neural Networks

Machine Learning and the Deuteron by Kebble and Rios and
Variational Monte Carlo calculations of $A \leq 4$ nuclei with an
artificial neural-network correlator ansatz by Adams et al.

Adams et al:

$$H_{LO} = - \sum_i \frac{\vec{\nabla}_i^2}{2m_N} + \sum_{i < j} (C_1 + C_2 \vec{\sigma}_i \cdot \vec{\sigma}_j) e^{-r_{ij}^2 \Lambda^2 / 4} \\ + D_0 \sum_{i < j < k} \sum_{\text{cyc}} e^{-(r_{ik}^2 + r_{ij}^2) \Lambda^2 / 4}, \quad (1)$$

where m_N is the mass of the nucleon, $\vec{\sigma}_i$ is the Pauli matrix acting on nucleon i , and $\sum_{\text{cyc}}$ stands for the cyclic permutation of i , j , and k . The low-energy constants C_1 and C_2 are fit to the deuteron binding energy and to the neutron-neutron scattering length

Deep learning neural networks, Variational Monte Carlo calculations of $A \leq 4$ nuclei with an artificial neural-network correlator ansatz by Adams et al.

An appealing feature of the neural network ansatz is that it is more general than the more conventional product of two- and three-body spin-independent Jastrow functions

$$|\Psi_V^J\rangle = \prod_{i < j < k} \left(1 - \sum_{\text{cyc}} u(r_{ij})u(r_{jk}) \right) \prod_{i < j} f(r_{ij}) |\Phi\rangle, \quad (2)$$

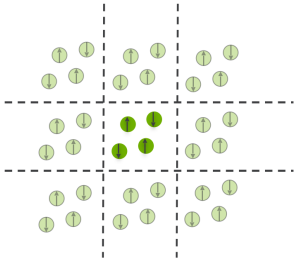
which is commonly used for nuclear Hamiltonians that do not contain tensor and spin-orbit terms. The above function is typically replaced by a deep Neural Network.

Ansatz for a fermionic state function, Jane Kim et al, Commun Phys 7, 148 (2024)

$$\Psi_T(\mathbf{X}) = \exp U(\mathbf{X})\Phi(\mathbf{X}).$$

1. Build in fermion antisymmetry for network compactness
2. Permutation-invariant Jastrow function improves ansatz flexibility
3. Build U and Φ functions from fully connected, deep neural networks
4. Use Slater determinant (or Pfaffian) Φ to enforce antisymmetry with single particle wavefunctions represented by neural networks

Nuclear matter setup

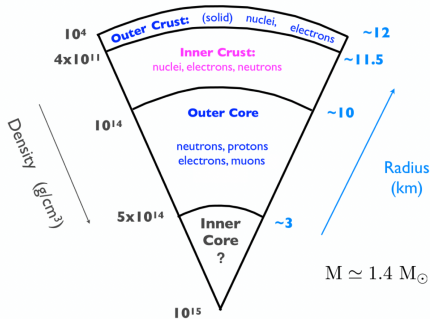


- Periodic boundary conditions and coordinate system

$$\mathbf{r}_i \longrightarrow \tilde{\mathbf{r}}_i = \left\{ \sin\left(\frac{2\pi}{L}\mathbf{r}_i\right), \cos\left(\frac{2\pi}{L}\mathbf{r}_i\right) \right\}$$

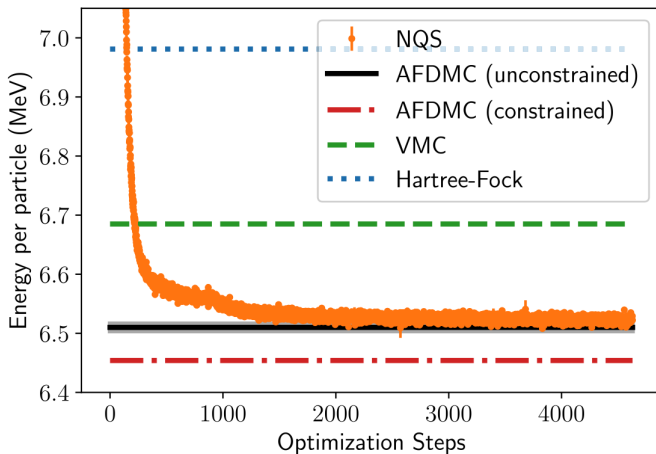
- Potential energy contribution from particle images
- Remove Coulomb potential

Neutron star structure

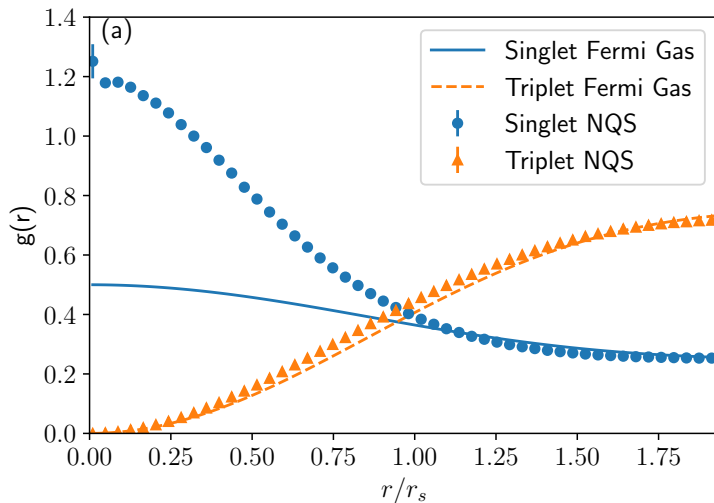


- Mostly neutrons but composition varies with density
- Nuclei in crust are squeezed into uniform matter in core
- Likely neutron superfluid in inner crust and outer core
- Calculations currently focus on inner crust

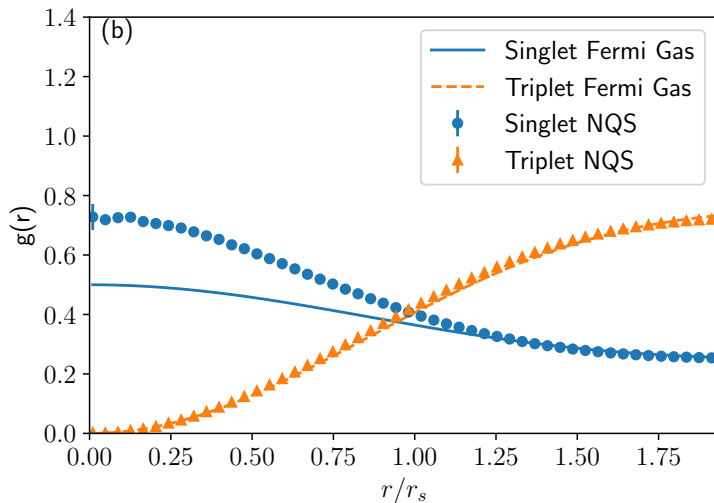
Dilute neutron star matter from neural-network quantum states by Fore et al, Physical Review Research 5, 033062 (2023) at density $\rho = 0.04 \text{ fm}^{-3}$



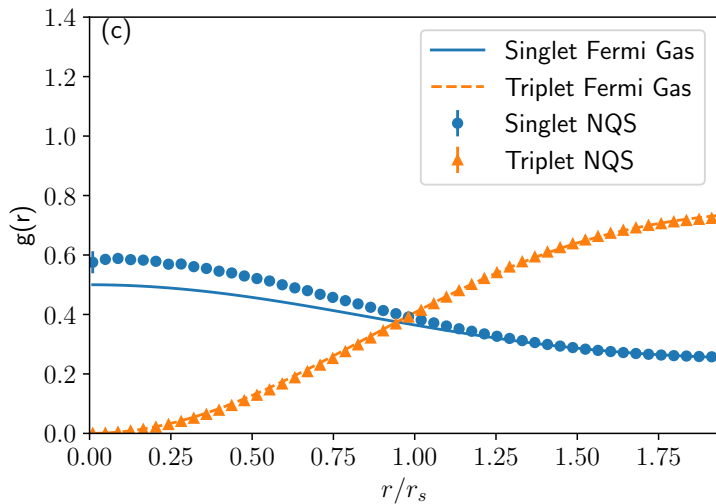
Pairing and Spin-singlet and triplet two-body distribution functions at $\rho = 0.01 \text{ fm}^{-3}$



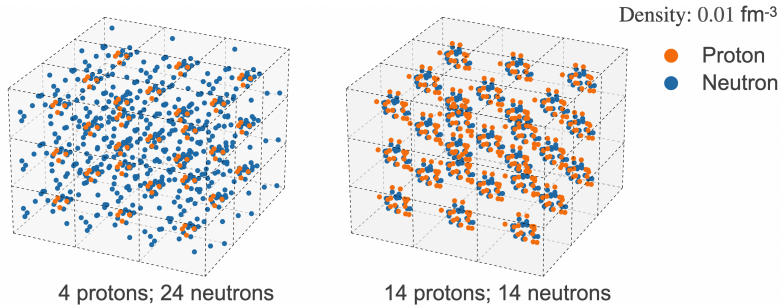
Pairing and Spin-singlet and triplet two-body distribution functions at $\rho = 0.04 \text{ fm}^{-3}$



Pairing and Spin-singlet and triplet two-body distribution functions at $\rho = 0.08 \text{ fm}^{-3}$

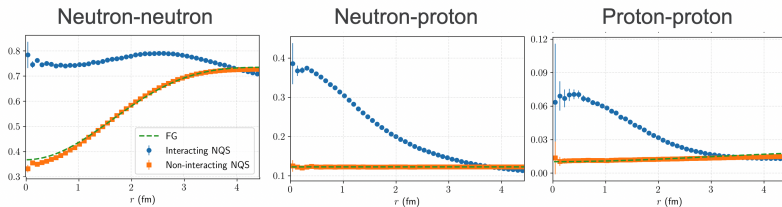


Self-emerging clustering



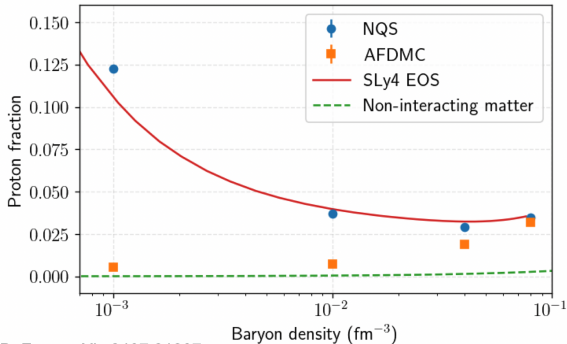
B. Fore, arXiv:2407.21207

Clustering: Two-body pair distributions



B. Fore, arXiv:2407.21207

Nuclear matter proton fraction

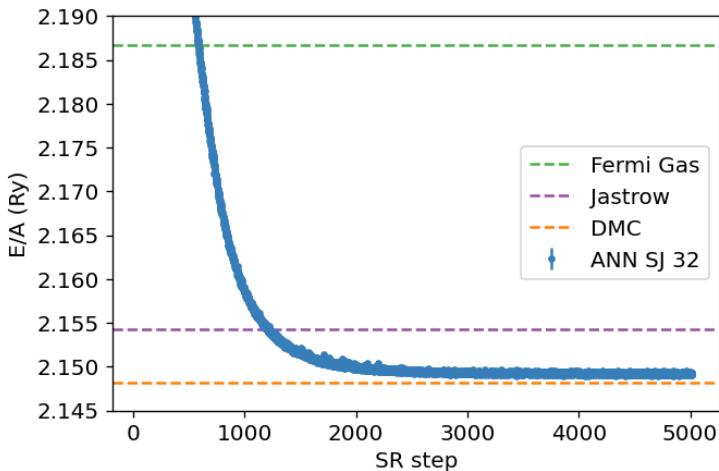


B. Fore, arXiv:2407.21207

Assumptions:

- Charge neutrality
 $n_p = n_e$
- Beta equilibrium
 $\mu_e = \mu_n - \mu_p$

The electron gas in three dimensions with $N = 14$ electrons (Wigner-Seitz radius $r_s = 2$ a.u.), Gabriel Pescia, Jane Kim et al. PRB 110, 035108 (2024),



Switching to quantum dots

The Hamiltonian of the quantum dot is given by

$$\hat{H} = \hat{H}_0 + \hat{V},$$

where $\hat{H}_0$ is the many-body HO Hamiltonian, and $\hat{V}$ is the inter-electron Coulomb interactions. In dimensionless units,

$$\hat{V} = \sum_{i < j}^N \frac{1}{r_{ij}},$$

with $r_{ij} = \sqrt{r_i^2 - r_j^2}$.

Separable Hamiltonian with the relative motion part ($r_{ij} = r$)

$$\hat{H}_r = -\nabla_r^2 + \frac{1}{4}\omega^2 r^2 + \frac{1}{r},$$

Analytical solutions in two and three dimensions for two electrons (M. Taut 1993 and 1994).

Quantum Dots as Artificial Atoms

- ▶ Semiconductor-confined electrons
- ▶ Tunable confinement ω
- ▶ Strong Coulomb correlations
- ▶ Few-body $\rightarrow$ mesoscopic regime

Goal:

Understand correlation-driven structure formation

Why Is This Hard?

- ▶ Exponential Hilbert space growth
- ▶ Strong correlation effects
- ▶ Near-Wigner crystallization at weak confinement

Traditional methods:

- ▶ Exact diagonalization (small N)
- ▶ Diffusion Monte Carlo
- ▶ Configuration interaction

Idea of Physics-Informed Neural Networks (PINNs)

Instead of sampling:

$$H\Psi = E\Psi$$

Minimize PDE residual:

$$\mathcal{L} = \langle |(H - E)\Psi_\theta|^2 \rangle$$

Advantages:

- ▶ No stochastic Monte Carlo noise
- ▶ Direct enforcement of Schrödinger equation
- ▶ Continuous representation

See demonstration of PINNs at [https:](https://github.com/CompPhysics/AdvancedMachineLearning/blob/main/doc/pub/week3/ipynb/BlackScholesPINN.ipynb)

[//github.com/CompPhysics/AdvancedMachineLearning/blob/main/doc/pub/week3/ipynb/BlackScholesPINN.ipynb](https://github.com/CompPhysics/AdvancedMachineLearning/blob/main/doc/pub/week3/ipynb/BlackScholesPINN.ipynb)

Wavefunction Ansatz

$$\Psi(R) = \text{SD}(R') \exp(U(R) + W_{\theta}(R))$$

Components:

- ▶ Slater determinant
- ▶ Jastrow correlation
- ▶ Neural correlator
- ▶ Backflow
- ▶ Permutation invariant construction

What is backflow?

Learned nonlocal transformations of coordinates

Backflow in neural quantum state calculations is a learned, nonlocal transformation of particle coordinates or features that allows the wavefunction to encode collective many-body correlations and accurate fermionic nodal structures far beyond mean-field or Jastrow forms.

CTNN backflow

CTNN backflow is a symmetry-preserving, convolutional realization of backflow in neural quantum states, where effective coordinates or features are generated by translation-invariant neural networks to encode collective many-body correlations and improve fermionic nodal structure efficiently.

PINNs and harmonic oscillator quantum dot

N	ω	DMC	PINN+BF	% err	PINN+CTNN	% err
2	0.001	—	0.0137948(8)	—	0.013778(1)	—
	0.01	—	0.69125(2)	—	0.69036(1)	—
	0.10	3.55385(5)	3.5549(1)	+0.0295	3.55388(5)	+0.0008
	0.50	11.78484(6)	11.7895(4)	+0.0395	11.7847(2)	-0.0000
	1.00	20.15932(8)	20.1610(6)	+0.0083	20.1585(3)	-0.0041
12	0.001	—	0.515823(3)	—	0.515365(4)	—
	0.01	—	2.48620(5)	—	2.47363(4)	—
	0.10	12.26984(8)	12.2731(2)	+0.0160	12.2718(1)	—
	0.50	39.1596(1)	39.1786(8)	+0.0485	39.1604(3)	+0.0018
	1.00	65.7001(1)	65.717(1)	+0.0257	65.69556(5)	-0.0069
20	0.001	—	—	—	1.293033(6)	—
	0.01	—	—	—	6.14645(5)	—
	0.10	29.9779(1)	—	—	29.9888(2)	+0.0364
	0.50	93.8752(1)	—	—	93.8789(5)	+0.0040
	1.00	155.8822(1)	—	—	155.8738(7)	-0.0024

Observations (or conclusions if you prefer)

- ▶ Need for AI/Machine Learning in physics, lots of ongoing activities
- ▶ To solve many complex problems and facilitate discoveries, multidisciplinary efforts are required involving scientists in physics, statistics, computational science, applied math and other fields.
- ▶ There is a need for focused AI/ML learning efforts that will benefit accelerator science and experimental and theoretical programs

More observations

- ▶ How do we develop insights, competences, knowledge in statistical learning that can advance a given field?
 - ▶ For example: Can we use ML to find out which correlations are relevant and thereby diminish the dimensionality problem in standard many-body theories?
 - ▶ Can we use AI/ML in detector analysis, experimental design, analysis of experimental data and more?
 - ▶ Can we use AI/ML to carry out reliable extrapolations by using current experimental knowledge and current theoretical models?
- ▶ The community needs to invest in relevant educational efforts and training of scientists with knowledge in AI/ML. These are great challenges to the CS and DS communities
- ▶ Quantum computing and quantum machine learning not discussed here
- ▶ Most likely tons of things I have forgotten

Possible start to raise awareness about ML in our own field

- ▶ Make an ML challenge in your own field a la [Learning to discover: the Higgs boson machine learning challenge](#).
Alternatively go to kaggle.com at <https://www.kaggle.com/c/higgs-boson>
- ▶ HEP@CERN and HEP in general have made significant impacts in the field of machine learning and AI. Something to learn from